

Development and Validation of a Portable Automotive RGB Lighting Diagnostic Platform Using the TLC6C5724-Q1 Automotive LED Driver with Local Control and MQTT-Based Remote Monitoring through Node-RED

Pratik Somnath Sukare

Faculty of Applied Natural Sciences and Cultural Studies

Ostbayerische Technische Hochschule Regensburg

Regensburg, Germany

pratik.sukare@st.oth-regensburg.de

Abstract—A modern automotive lighting solution requires the availability of an LED controller along with the ability to detect faults that can occur. This project presents a portable testing and control device for automotive lighting solutions that employs the Texas Instruments TLC6C5724-Q1, a 24-channel constant-current RGB LED driver.

An ESP32 microcontroller is used to control the driver, perform diagnostic tests, operate the RGB LEDs, and provide a local TFT display with buttons for local interaction. The system was tested under normal operating conditions and controlled fault conditions to verify the implemented control and diagnostic functions. The final setup provides a compact and practical platform for studying LED driver operation, communication, diagnostics, and fault detection.

I. INTRODUCTION

Automotive lighting systems utilize multichannel LED drivers to manage the luminous intensity and color in numerous outputs. In addition to controlling the current and implementing pulse width modulation, it is necessary for these drivers to detect electrical and thermal faults that may affect the functioning of the LED lighting. Integrated diagnostics, thus, becomes an essential feature of automotive LED control.

The Texas Instruments TLC6C5724-Q1 is a 24-channel constant-current RGB LED driver designed for automotive applications. This device offers grayscale and current control via a serial interface; it also provides diagnostic capabilities to detect open and short LED conditions, adjacent-pin shorts, reference resistor faults, and thermal issues [1]. This allows the use of this device not only for LED control but also for fault detection.

The ESP32 is used to control all 24 output channels, perform diagnostic procedures, and provide local control via a TFT display and push buttons. Remote monitoring and control are provided via the MQTT protocol, with Eclipse Mosquitto as the broker and Node-RED as the browser-based user interface [2]–[5].

The system was tested in the conditions of normal operation, fault introduction in a controlled manner, automatic LED tests, and network reconfiguration. As a result, the portable

platform providing local and remote control, diagnostics, and test capabilities for TLC6C5724-Q1 was created.

II. SETUP OVERVIEW

The structure of the project consists of four layers:

- 1) TLC6C5724-Q1 LED driver,
- 2) ESP32 controller,
- 3) local human-machine interface (TFT), and
- 4) laptop-based remote interface (Node-Red).

Both the firmware and the dashboard provide access to the controlling and diagnostic parameters for all 24 output channels.

The ESP32 sends the TLC6C5724-Q1 LED driver's TLC serial control commands and provides the Grayscale Clock and Blanking control for PWM functionality. The ERR signal serves as the fault indicator. A TFT display and three push buttons provide local navigation, parameter selection, manual control, and access to diagnostics when the network is unavailable.

On the remote side, the laptop runs Eclipse Mosquitto and Node-RED. The ESP32 connects to the laptop hotspot, publishes status and diagnostic results, and subscribes to command topics. Node-RED uses the same broker to display device information and send commands back to the ESP32. A Windows batch launcher starts Mosquitto and Node-RED, waits until the server is available, and then automatically opens the diagnostics page.

III. DESIGN AND THEORY

A. TLC6C5724-Q1 LED Driver

The TLC6C5724-Q1 is a 24-channel constant-current RGB LED driver. Each output can sink up to 50mA. The device provides 7-bit dot correction for individual channels and 8-bit brightness control for the RGB groups. Grayscale control can be configured for 12-, 10-, or 8-bit resolution [1]. These functions support channel adjustment and PWM brightness control.

The driver also includes several diagnostic functions. These include LED open and short detection, output short-to-ground

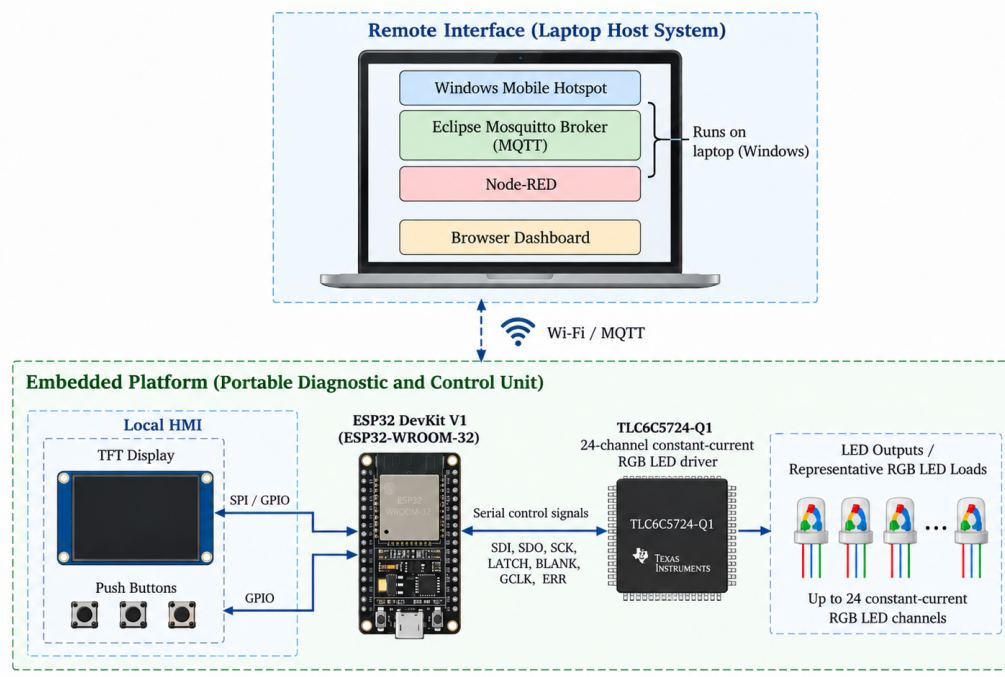


Fig. 1: System architecture of the portable TLC6C5724-Q1 diagnostic and control platform.

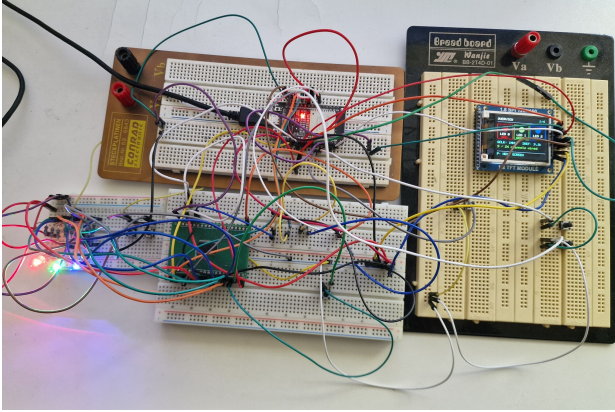


Fig. 2: Prototype overview showing the ESP32 controller, TLC6C5724-Q1 hardware, TFT interface, push buttons, wiring, and RGB LED outputs.

detection, adjacent-pin short detection, thermal prewarning and shutdown, and IREF resistor open and short detection. It also monitors GCLK and LOD/LSD register integrity and provides an LOD/LSD circuit self-test [1]. These functions form the basis of the diagnostic pages on the TFT and Node-RED dashboard.

B. ESP32–TLC Hardware Interface

The ESP32 acts as the main controller and network interface. Communication with the TLC6C5724-Q1 uses an SPI based for control and diagnostic data. Separate signals are used for latching, PWM timing and fault indication. This keeps

the hardware interface simple and separates data transfer from LED timing and diagnostics.

C. Firmware Structure and Main Loop

The firmware runs from the main loop and handles each task in small steps. Long delays are avoided so that LED tests, button input, diagnostics, Wi-Fi, MQTT, and TFT updates can all run regularly.

MQTT messages are first checked and stored as pending commands. The main loop then performs the requested TLC action. This keeps network handling short and allows the local controls to respond quickly. Automatic LED tests use a simple state machine. The current test, step, direction, and elapsed time decide when the next LED pattern is sent.

D. Diagnostic Functions

The diagnostic functions implemented in the project are summarized in Table I [1].

E. Local TFT and Button Interface

The local interface uses a compact colour 128×160 SPI TFT and three push buttons. The buttons provide page navigation, option selection, and execution of the selected function. The local TFT interface is independent of MQTT availability, allowing the LED-driver hardware to remain controllable if a wireless connection is unavailable.

F. Remote Monitoring and Control

The ESP32 connects to the laptop over Wi-Fi. MQTT is used to send commands and receive status data [2], [6]. Mosquitto runs on the laptop and handles the MQTT messages [7].

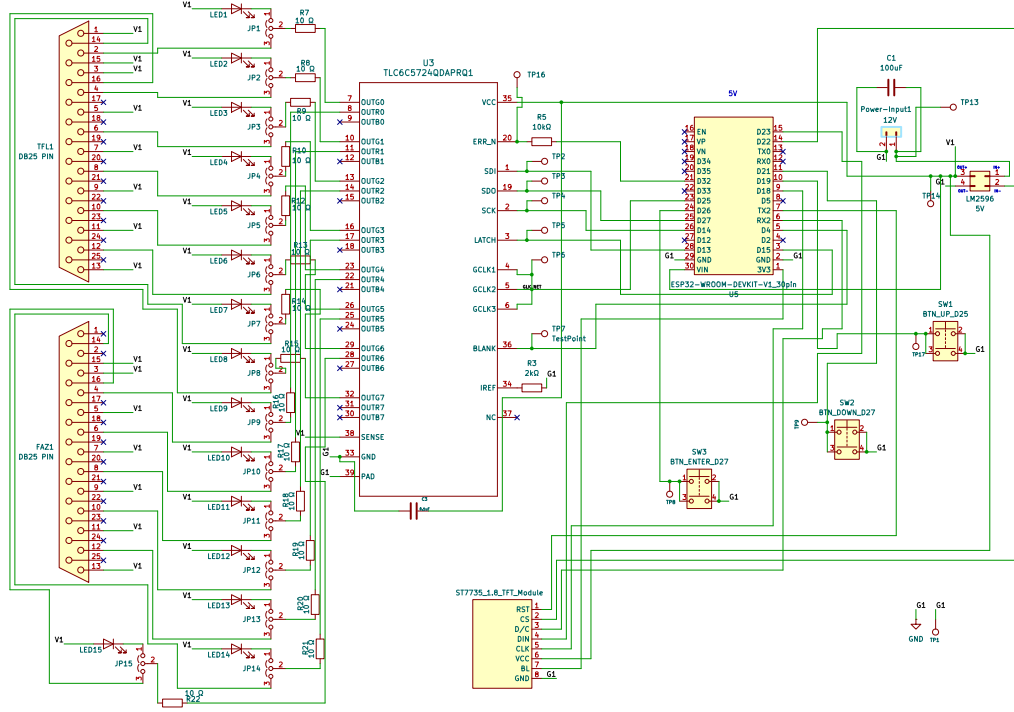


Fig. 3: KiCad schematic of the ESP32–TLC6C5724-Q1 prototype showing the driver interface, power supply, IREF network, local controls, display connection, and LED output wiring.

TABLE I: Implemented Diagnostic Functions

Function	Purpose
ERR monitor	Monitors the active fault signal from the TLC6C5724-Q1
Device status	Reads the main internal diagnostic status of the driver
Thermal error (TEF)	Indicates an over-temperature fault condition
Thermal warning (PTW)	Indicates that the device temperature is approaching the warning limit
IREF open (IOF)	Detects an open condition in the IREF resistor path
IREF short (ISF)	Detects a short condition in the IREF resistor path
Channel scan	Checks the output channels one by one for LED faults
LED open (LOD)	Reports an open-load condition on the scanned output
LED short (LSD)	Reports a short condition on the scanned output
APS test	Checks for a short between adjacent output pins
LOD/LSD self-test	Checks the internal open- and short-detection circuits
NEG/GCLK test	Checks GCLK operation and negate-bit integrity
Error clear	Clears stored diagnostic flags before a new test
Raw SID readout	Displays the raw diagnostic status data for debugging

TABLE II: Implemented Automatic LED Tests

Test	Purpose
RGB colour cycle	Cycles the complete RGB colour states
RGB phase rotation	Rotates R/G/B assignments across the RGB output groups
Sequential chase	Steps through the RGB output channels in sequence
Brightness sweep	Smooth white fade with OFF dwell
All white	Static functional illumination test
All off	Static output-off test

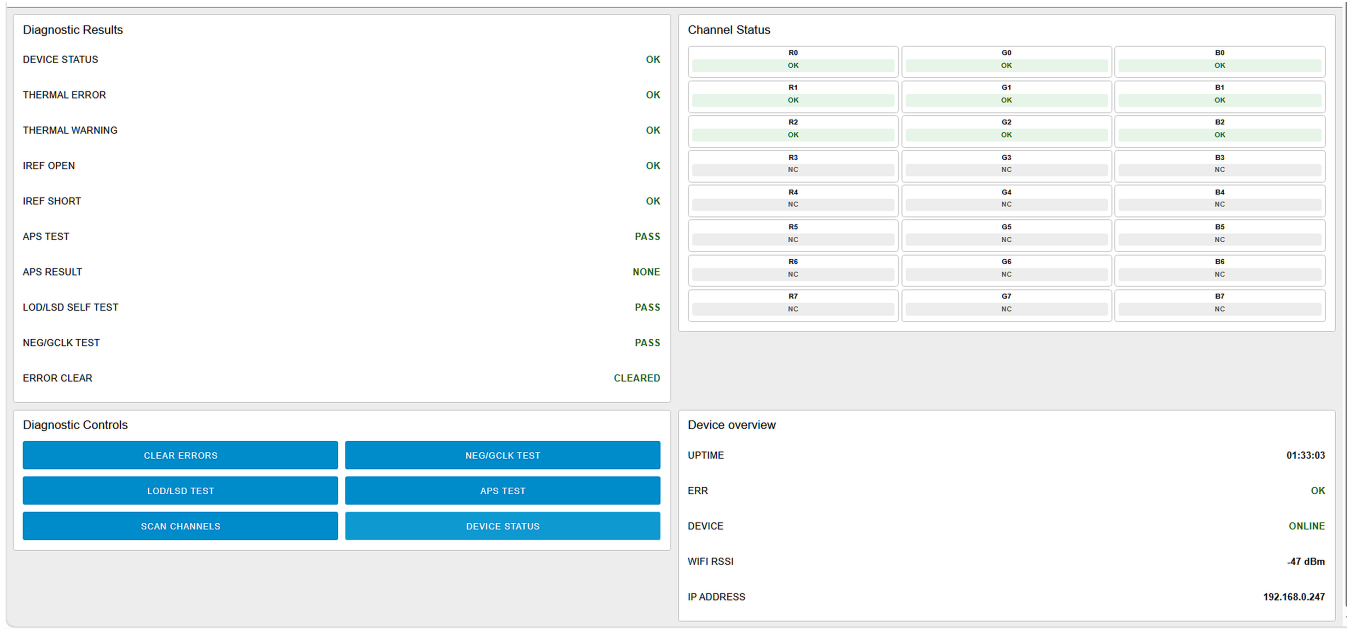
Node-RED provides the browser dashboard [4]. It shows device status and diagnostic results. It also provides manual LED control and automatic test controls using Dashboard 2 [5].

G. Automatic LED Demonstration Tests

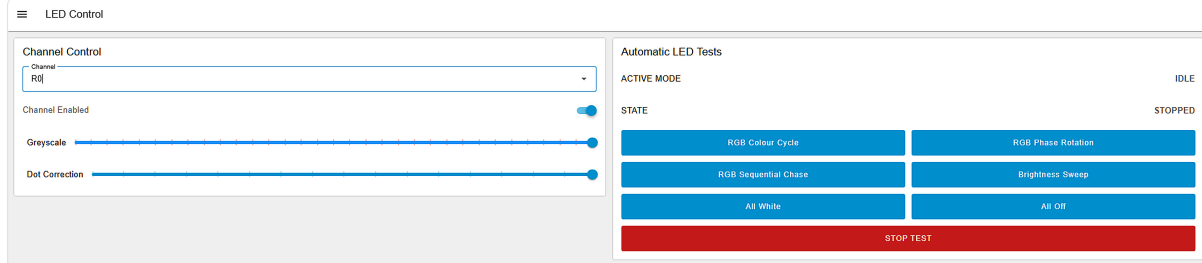
Six automatic LED tests were implemented for functional testing and demonstration. The tests are summarized in Table II.

H. Portable Startup and Network Recovery

The remote setup does not depend on one fixed Wi-Fi network. The laptop provides a Windows mobile hotspot for the ESP32, while Mosquitto and Node-RED run locally on the laptop. A launcher script starts both services, waits for Node-



(a) Diagnostics page showing device status, channel status, diagnostic controls, and device overview.



(b) LED control page for manual channel settings and automatic LED tests.

Fig. 4: Node-RED Dashboard 2 remote monitoring and control interface.

RED to become available, and then opens the diagnostics dashboard.

The firmware can recover when the network changes. If the Wi-Fi connection is lost, the ESP32 tries the saved network profiles again. If Wi-Fi remains connected but the MQTT broker cannot be reached for a set time, the ESP32 disconnects and returns to the preferred hotspot [6].

IV. RESULTS

The system was tested under normal operating conditions and under controlled fault conditions. The diagnostic functions correctly detected LED open and short conditions, short-to-ground faults, adjacentpin shorts using APS, and IREF open and short faults. The device-status check, LOD/LSD self-test, NEG/GCLK test, and error-clear function also returned the expected results. Together, these tests confirm that the platform can identify both channel-level and device-level faults.

Remote control through Node-RED was also verified. Diagnostic commands, manual LED control, automatic LED tests, and device status could be operated from the dashboard. The launcher and Wi-Fi/MQTT recovery functions also worked as intended.

V. CONCLUSION

A portable automotive diagnostic system using the automotive LED driver TLC6C5724-Q1 and ESP32 microcontroller was designed for automotive lighting solutions. This project combined LED diagnostics, individual control, group based control and both local and remote monitoring and control. Several fault conditions were tested under controlled conditions and the diagnostic functions identified the expected faults. The results were displayed in realtime on both interfaces. The fault status could also be cleared locally or remotely after the fault condition was removed. The same setup can support different automotive lighting applications such as headlights, interior lighting, rear lighting, ambient lighting and advanced solutions like window lighting and puddle lighting.

ACKNOWLEDGMENT

I would like to thank Mr. Gerald Schickhuber and Prof. Johannes Reschke for their guidance, technical feedback, and support during the development and validation of the project.

REFERENCES

- [1] *TLC6C5724-Q1 Automotive 24-Channel, Full Diagnostics, Constant-Current RGB LED Driver*, Texas Instruments, 2018, datasheet, Rev. A. [Online]. Available: <https://www.ti.com/lit/ds/symlink/tlc6c5724-q1.pdf>
- [2] “MQTT Version 5.0,” OASIS Open, OASIS Standard, Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [3] Eclipse Mosquitto Project, “Eclipse Mosquitto Documentation,” Online documentation, accessed 2026-09-09. [Online]. Available: <https://mosquitto.org/documentation/>
- [4] OpenJS Foundation and Contributors, “Node-RED,” Online documentation, accessed 2026-09-09. [Online]. Available: <https://nodered.org/>
- [5] FlowFuse, “Node-RED Dashboard 2.0 Documentation,” Online documentation, accessed 2026-09-09. [Online]. Available: <https://dashboard.flowfuse.com/getting-started>
- [6] Espressif Systems, “Arduino ESP32 Wi-Fi API,” Online documentation, accessed 2026-09-09. [Online]. Available: <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/wifi.html>
- [7] Eclipse Mosquitto Project, “mosquitto.conf Manual Page,” Online documentation, accessed 2026-09-09. [Online]. Available: <https://www.mosquitto.org/man/mosquitto-conf-5.html>